

From Strategy Recognition to Strategic Adaptation: Validating LLM Agents in Spatial Voting Models

Keywords: computational social science, large language models, agent-based modeling, agent validation, party competition

Extended Abstract

Political parties’ strategic positioning is a central mechanism of electoral competition, and spatial models have provided a tractable way to study how behavioral rules generate collective political dynamics. In the classic work of Downs (1957), parties are modeled as vote-seeking competitors who choose policy positions in an ideological space to maximize voter support, while voters select the party closest to their own positions. Laver (2005) extended the classic framework of spatial models in an agent-based model where parties in a two-dimensional policy space, each representing economic and social dimensions, follow one of five adaptive strategies: Sticker (never moves), Predator (chases the largest party), Aggregator (moves toward supporter centroid), Hunter (repeats moves that gain votes), and Random (moves randomly). Laver’s model allows the observation of how different strategies in political competitions produce distinct movement patterns and vote trajectories, and more importantly, it enables the evaluation of simulated agents on whether they can recover and enact known mechanisms of strategic interactions (Figure 1).

As computational social science increasingly experiments with LLM-based agents to simulate social and political actors Park et al. (2023); Argyle et al. (2023); Horton (2023), a fundamental question arises: can LLMs accurately recognize the strategic behaviors of political parties from observational data, and can they themselves act as effective strategic agents in competitive electoral settings? Answering these questions is critical because LLMs that cannot recognize mechanisms are unlikely to implement strategic adaptation.

To tackle our research questions, we present two complementary studies that address these questions within Laver’s simulation framework. Study 1 tests whether LLMs can *infer* party strategies from time-series data alone. Study 2 tests whether an LLM can *adopt* effective strategies as a competing party agent embedded in the simulation.

Study 1—Mechanism Recognition. We generated 50 simulations of the Laver model, each running 100 time steps with 3–7 parties and 500–1,000 voters, producing 254 individual party–strategy instances distributed across the five strategies. We evaluated six LLMs: GPT-4o, MiniMax, Qwen2.5-7B, Qwen3-30B, Qwen3-1.7B, and Qwen3-0.6B. Each model received sampled time-series data on party positions and vote counts and was asked to classify every party’s strategy without access to labels. To manage context length, we provided a 30-step summary of positions and votes from evenly spaced windows across each simulation.

The performance across models varies substantially, as GPT-4o achieves the highest overall accuracy at 39.0%, nearly double the random baseline of 20% and substantially above the second-best model, MiniMax (25.2%). A clear strategy difficulty hierarchy emerges (Figure 2): Sticker is near-trivially detectable (100% for GPT-4o) due to zero positional variance, while

Predator is hardest ($\leq 12.1\%$ for all models) because it requires cross-party relational reasoning through tracking which party is largest and whether movement targets it. The GPT-4o confusion matrix (Figure 3) reveals systematic error patterns: Predator, Aggregator, and Hunter instances are predominantly misclassified as Hunter, suggesting models default to a “movement-toward-something” heuristic without distinguishing the target. Plus, model scale alone does not predict performance. In our implementation, Qwen3-30B, the largest open-source model tested, scores lowest at 9.4%, indicating that architecture and training regime matter more than parameter count for this task.

Study 2—Strategic Adaptation. We then deployed GPT-4o as an active party agent within the same simulation framework, competing against four rule-based opponents. Every three time steps, the LLM agent received its current position, vote share, and opponents’ positions, then selected from the five Laver strategies for its next moves. We tested six matchup conditions: LLM versus four identical-strategy opponents (Sticker, Predator, Aggregator, Hunter, or Random) and a mixed-strategy condition, with 3 replications per matchup over 50 time steps with 500 voters.

The LLM agent performs best against disorganized opponents: it exceeds the 20% equal-share baseline against Random (31.0%) and Predator (33.6%) opponents, performs near baseline against Sticker (20.5%) and Hunter (24.9%), but falls below baseline against Aggregator (18.2%) and in the mixed condition (15.1%; Figure 4). Critically, the LLM overwhelmingly converges on the Aggregator strategy regardless of opponent type, selecting it in 83–98% of decisions across all matchups (Figure 5). This rigid preference is effective against opponents that leave the voter center uncontested, but it fails against Aggregator opponents competing for the same centrist voters and in mixed environments requiring adaptive responses. The LLM’s poor performance in the mixed condition (15.1%, well below the 20% baseline) is particularly revealing: real political competition involves heterogeneous opponents, and the inability to adapt strategy to a diverse competitive landscape represents a significant limitation for simulation applications.

Cross-Study Implications. Comparing GPT-4o’s inference accuracy (Study 1) with its strategy usage frequency (Study 2) across the five strategies reveals a disconnect: the LLM’s strategic repertoire is narrow regardless of its recognition ability (Figure 6). It gravitates toward Aggregator even though it can identify Sticker successfully and struggles to recognize the very strategy it overwhelmingly adopts. These findings suggest that while LLMs can partially identify strategic patterns from observational data, they lack the adaptive, opponent-contingent reasoning required for robust political agency. For the growing use of LLM agents in computational social science simulations, our results indicate that passing an inference test is necessary but insufficient, as effective agency demands flexible strategic adaptation that current models do not yet exhibit. We argue that validation benchmarks for LLM-based social simulation agents should test both recognition and behavioral competence in adversarial settings, rather than relying only on survey-style alignment with human responses.

References

- Downs, A. (1957). *An Economic Theory of Democracy*. Harper & Row.
- Laver, M. (2005). Policy and the Dynamics of Political Competition. *American Political Science Review*, 99(2):263–281.

- Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th ACM Symposium on User Interface Software and Technology (UIST)*.
- Argyle, L. P., Busby, E. C., Fulda, N., Gubler, J. R., Rytting, C., & Wingate, D. (2023). Out of One, Many: Using Language Models to Simulate Human Samples. *Political Analysis*, 31(3):337–351.
- Horton, J. J. (2023). Large Language Models as Simulated Economic Agents: What Can We Learn from Homo Silicus? *NBER Working Paper 31122*.

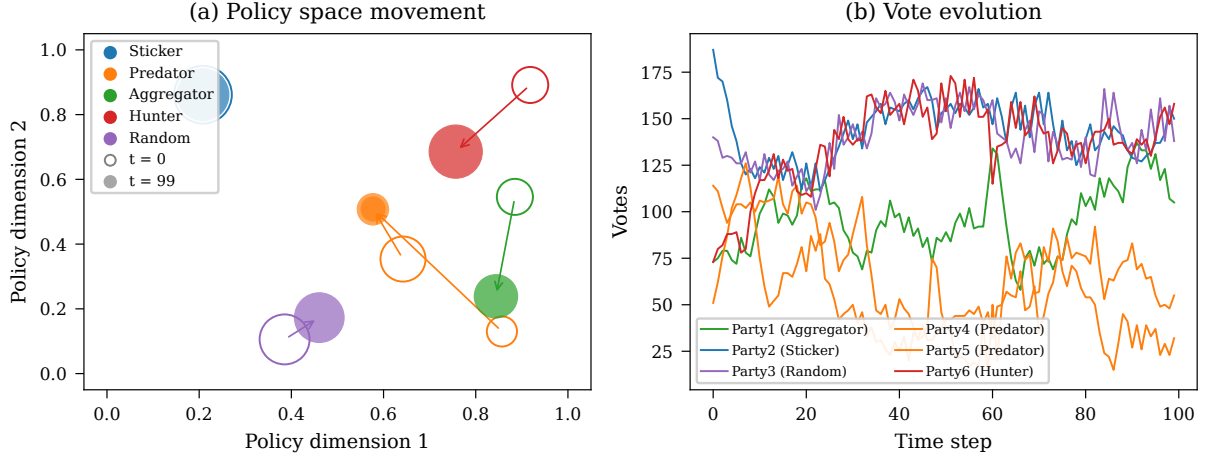


Figure 1: Example simulation (6 parties, 5 strategies). Left: party movement trajectories in 2D policy space from $t=0$ (open circles) to $t=99$ (filled circles). Right: corresponding vote count evolution. This illustrates the Laver (2005) framework used in both studies.

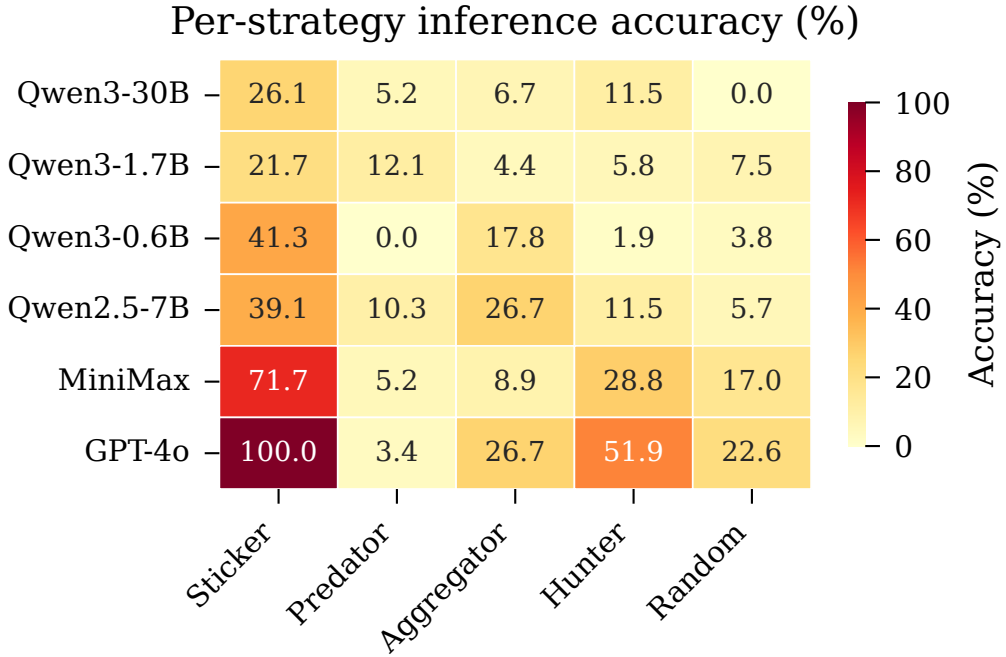


Figure 2: Per-strategy inference accuracy (%) across six LLMs (Study 1). Sticker is near-trivially detectable due to zero positional movement, while Predator is hardest across all models ($\leq 12.1\%$), requiring cross-party relational reasoning. Model scale does not predict performance: Qwen3-30B scores lowest overall.

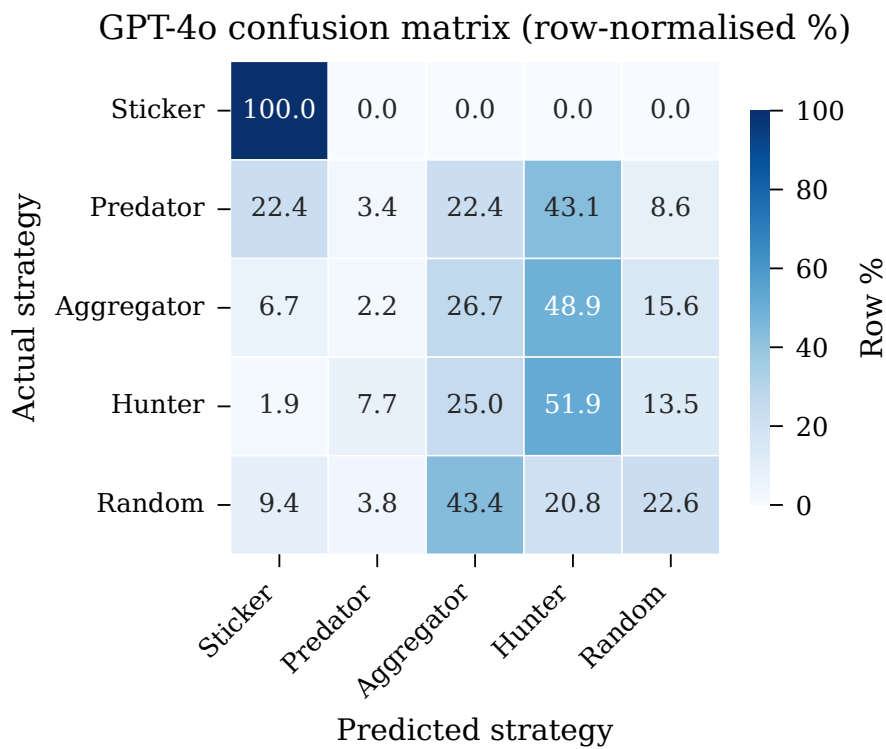


Figure 3: GPT-4o row-normalized confusion matrix (Study 1). Predator, Aggregator, and Hunter instances are predominantly misclassified as Hunter, revealing a default “movement-toward-something” heuristic. Random is most often confused with Aggregator.

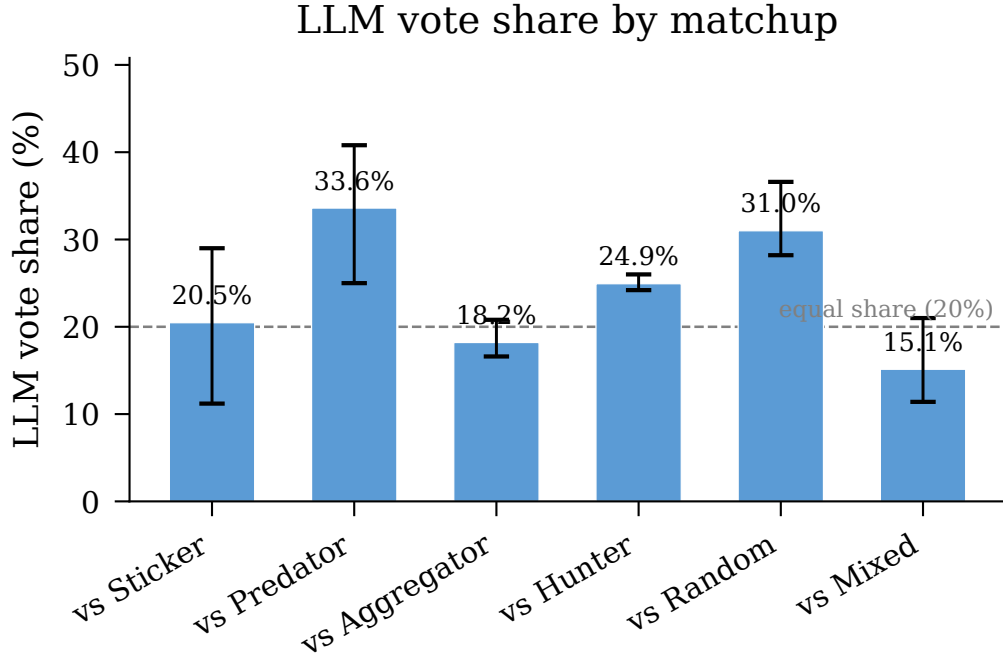


Figure 4: LLM agent vote share by matchup condition (Study 2). The dashed line indicates the 20% equal-share baseline. The LLM exceeds baseline against Predator and Random opponents but underperforms against Aggregator and in mixed environments. Error bars show standard deviation across 3 replications.

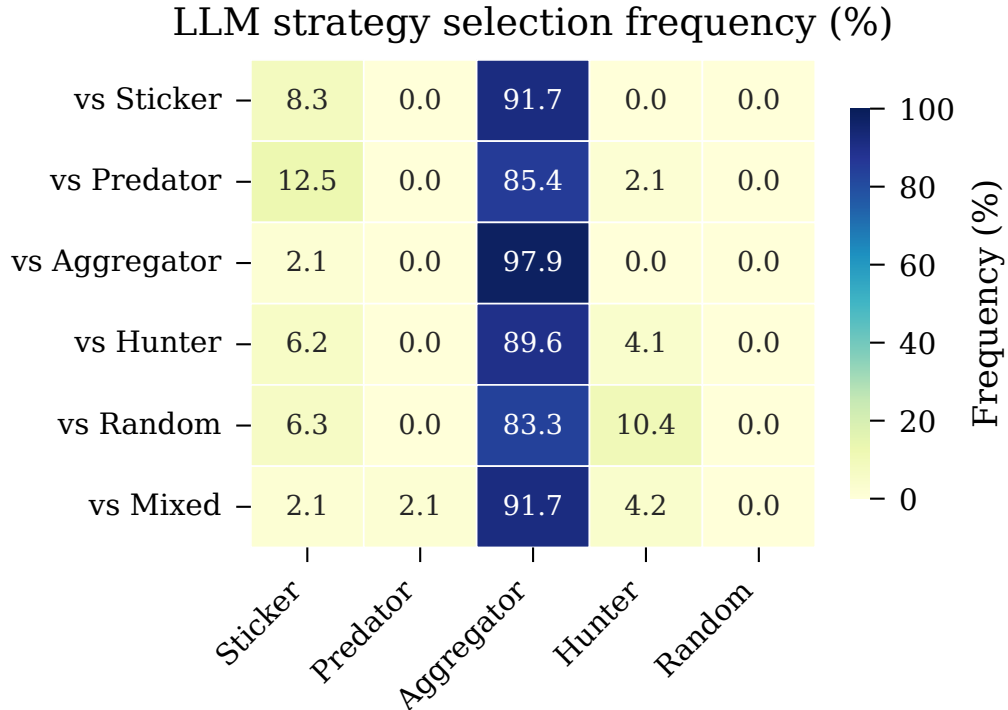


Figure 5: LLM strategy selection frequency across matchup conditions (Study 2). The LLM overwhelmingly selects Aggregator (83–98% of decisions) regardless of opponent type, showing minimal strategic adaptation.

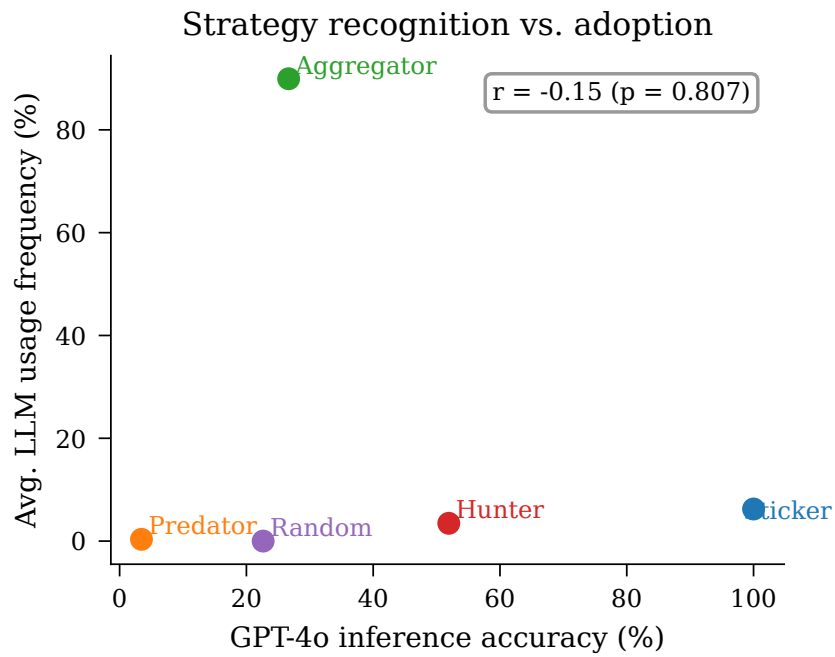


Figure 6: Cross-study comparison: GPT-4o’s inference accuracy (Study 1, x -axis) vs. average strategy usage frequency (Study 2, y -axis) for each of the five strategies. Aggregator dominates usage despite moderate recognition accuracy ($r = -0.15$, $p = 0.807$), revealing that the LLM’s strategic repertoire is disconnected from its recognition ability.